

# CachePool: Many-core cluster of customizable, lightweight scalar-vector PEs for irregular L2 data-plane workloads

Integrated Systems Laboratory (ETH Zürich)

**Zexin Fu, Diyou Shen**

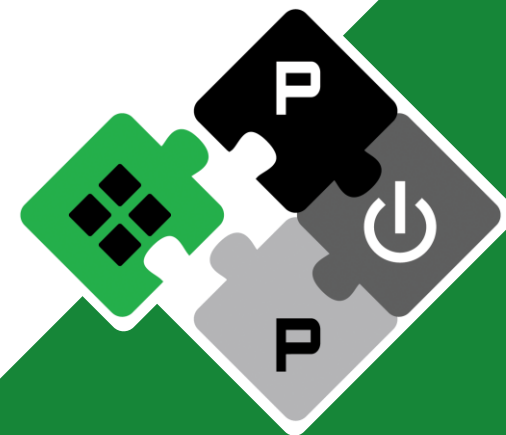
zexifu, dishen@iis.ee.ethz.ch

**Alessandro Vanelli-Coralli**  
**Luca Benini**

avanelli@iis.ee.ethz.ch  
lbenini@iis.ee.ethz.ch

**PULP Platform**

Open Source Hardware, the way it should be!



@pulp\_platform



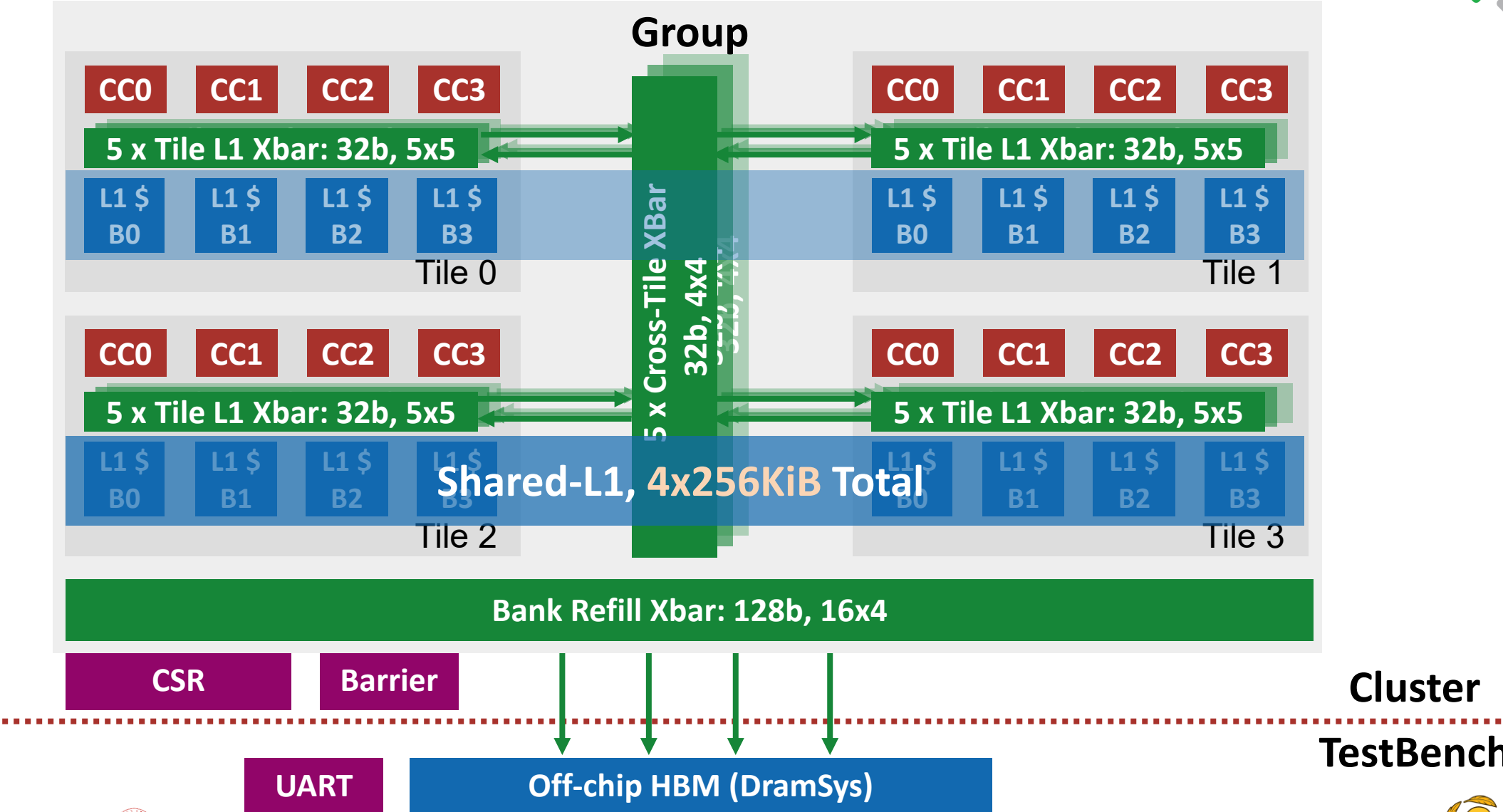
pulp-platform.org



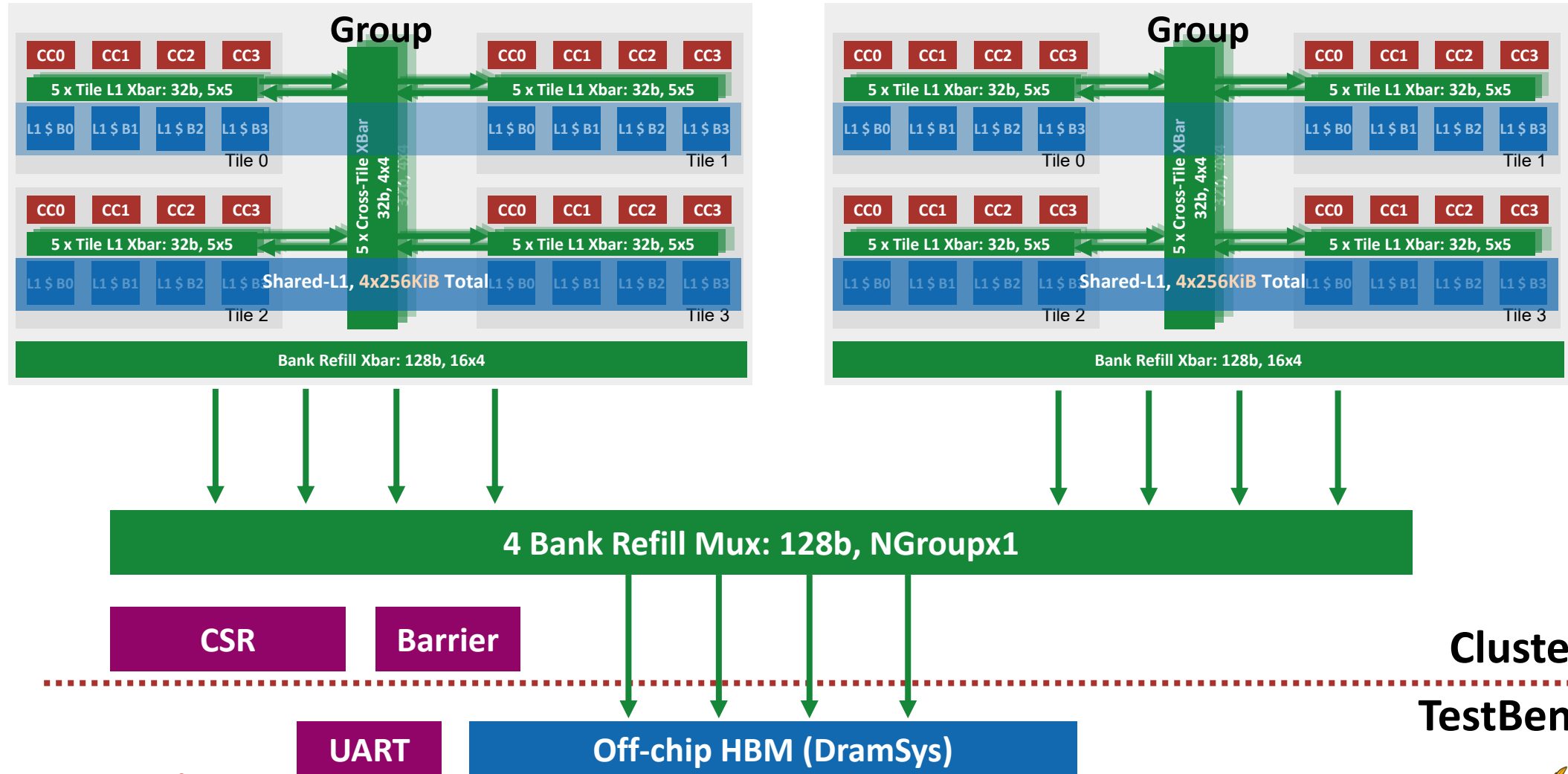
youtube.com/pulp\_platform



# Hardware Development (Cluster)



# Hardware Development (Cluster)



# Hardware Development (Cluster)



- **Tile and Group**

- Move refill xbar into the group level
- Adjust the interconnection to add remote group
- Add remote-group ports

- **Group NoC Wrapper**

- Add a new level group\_noc\_wrapper to instantiate the group
- This block will add some mux/xbar to connect to the NoC router (WIP)

- **Cluster**

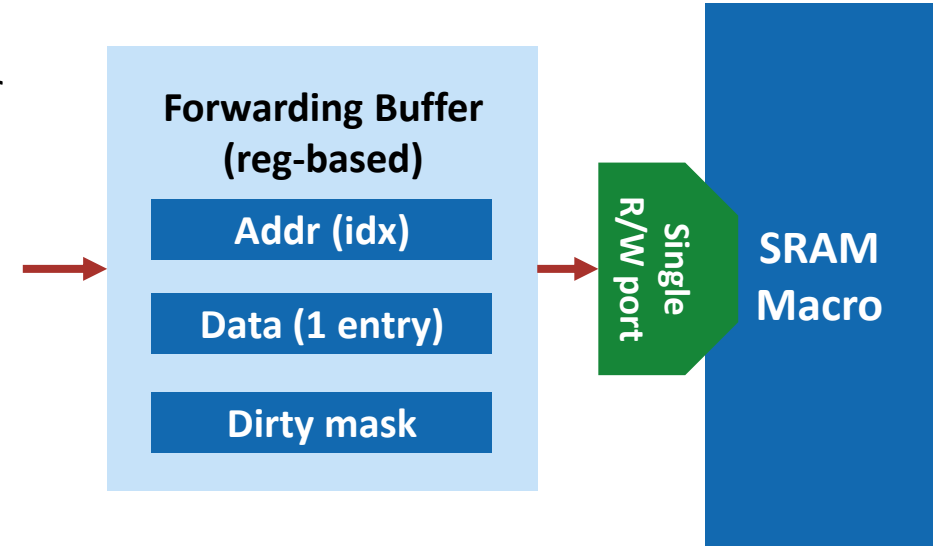
- Instantiate multiple group wrapper (tested 2 and 4)
- Temporarily use AXI MUX for connecting refilling ports from different groups to DRAM
- Now, we can execute some kernels without global edits (gemv, gemm)



# Hardware Development (L1 Cache)



- Forwarding Buffer implementation
  - Per-(way, bank) single-entry cache sitting between cache controller and SRAM
  - Reg-based, byte-mask-granular, with part-bitmap tracking for sub-line residency
  - Added handshake between the forwarding buffer and \$ ctrl
    - If the buffer is writing back
      - backpressure the coming req 1 cycle
    - If the buffer hit
      - the SRAM access can finish at the same cycle
  - Single preread populates all parts of the cache line on miss

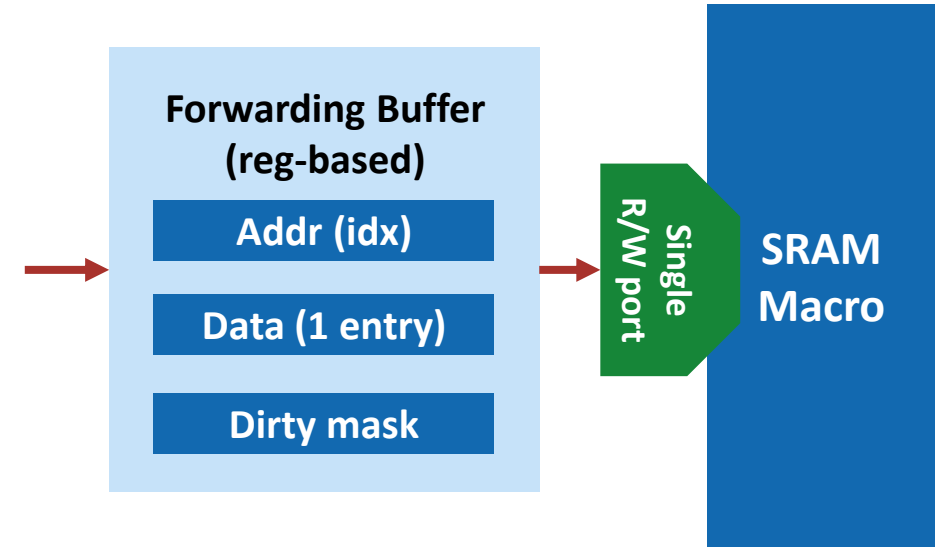


# Hardware Development (L1 Cache)



- Forwarding Buffer Features

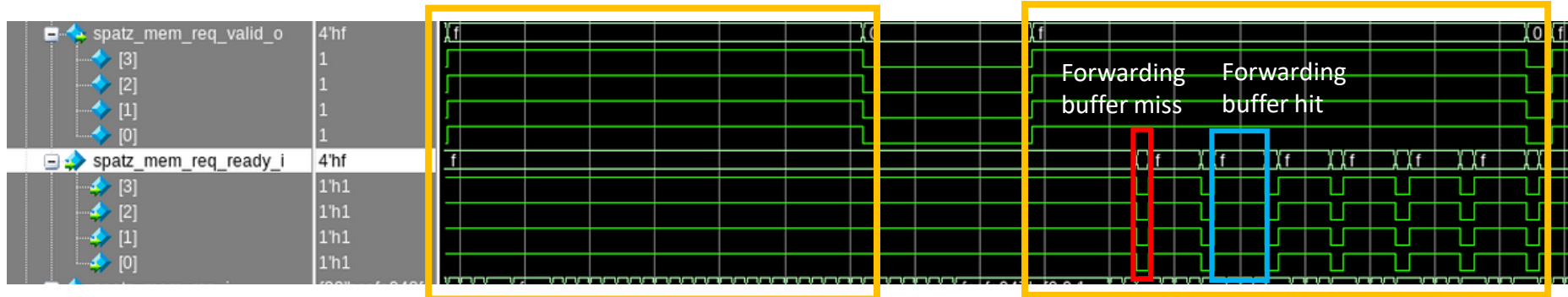
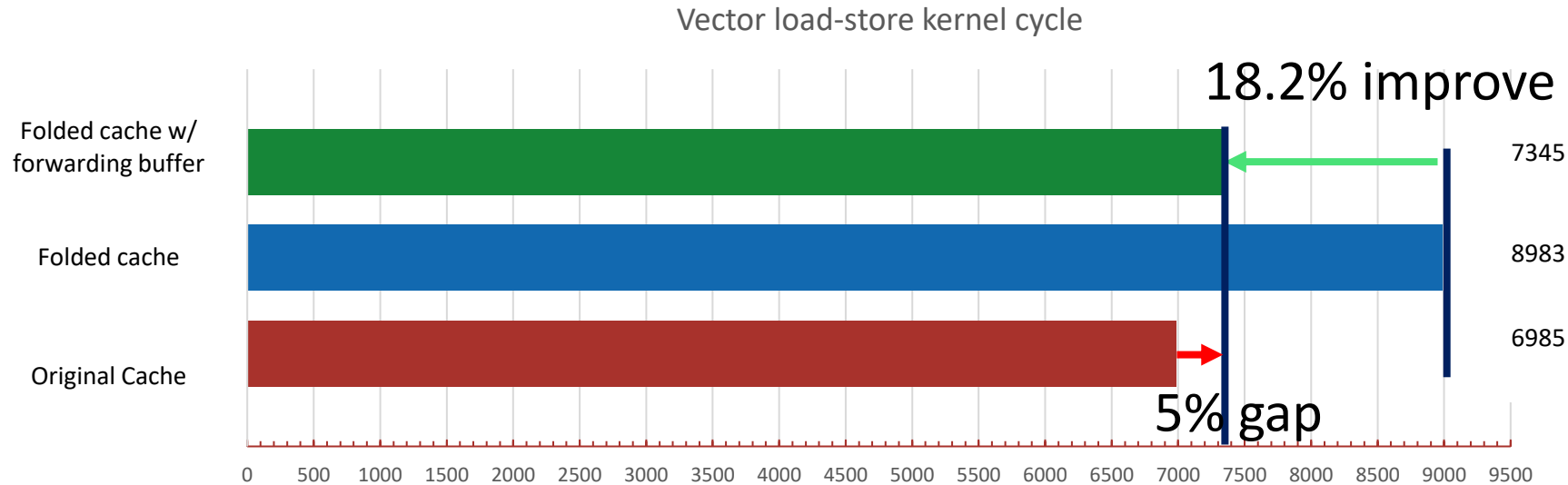
- **Read-hit serving:** same-line/same-part reads return from buffer; no SRAM access
- **Write absorption:** same-line writes merged into buffer; no SRAM access (write-back at replacement/idle cycle)
- **Part bitmap tracking:** per-part (16B) valid bit, supports incremental same-line accumulation
- **Read-write-merge:** preserves dirty parts when a refill + concurrent same-line write happen at the same cycle
- **Speculative writeback:** opportunistic flush during idle SRAM cycles + addr-transition prediction
- **Sticky victim entry:** buffer holds clean lines past their last write so re-reads still hit
- **RAW forwarding:** same-cycle read+write returns post-write merged data



# Hardware Development (L1 Cache)



- Forwarding Buffer Performance



Vector load (full bw)

Vector store (80% bw)



# Hardware Development (L1 Cache)



- Forwarding Buffer Performance
  - RLC kernel: 52465 → 37550 (28.4% improvement)
    - Pseudo Random replacement
    - Forwarding buffer





# Hardware Development (L1 Cache)



- Forwarding Buffer Power
  - SRAM access reduction (w/o vs. w/ forwarding buffer)
    - vector-rw      74K → 31K (-58%)
    - RLC kernel    62K → 33K (-47%)
  - It can be used to estimate the dynamic power reduction of SRAMs



# Next Step



- **InSitu-Cache Paper**

- Both Zexin and Diyou will focus on the InSitu cache paper in May (ICCD). The progress on other side might be delayed

- **Diyou Side**

- 7nm Flow (ongoing)
- RTL development on multi-group  
Add router connection and router mesh

- **Zexin Side**

- Rebase the folded cache+forwarding buffer branch to the multi-tile main branch
- Continue the insitu cache GVSoC model
- Fix the new RLC kernel bugs



# Timeline



|                   |                                                                  | 2025 |    |   |   |   |   |   | 2026 |   |   |   |    |    |
|-------------------|------------------------------------------------------------------|------|----|---|---|---|---|---|------|---|---|---|----|----|
|                   |                                                                  | 11   | 12 | 1 | 2 | 3 | 4 | 5 | 6    | 7 | 8 | 9 | 10 | 11 |
| System            | Multi-Tile Scaling<br>[RTL] <b>Complete</b>                      |      |    |   |   |   |   |   |      |   |   |   |    |    |
|                   | 4/16-Group Scaling<br>[GVSoC + RTL] <b>Ongoing</b>               |      |    |   |   |   |   |   |      |   |   |   |    |    |
| L1 Access Latency | Hardware optimization (Cache & Interco)<br>[RTL] <b>Clean up</b> |      |    |   |   |   |   |   |      |   |   |   |    |    |
|                   | Cache partitioning<br>[RTL] <b>Complete</b>                      |      |    |   |   |   |   |   |      |   |   |   |    |    |
|                   | WT-L1 by master student<br>[RTL] <b>Partial-Complete</b>         |      |    |   |   |   |   |   |      |   |   |   |    |    |
| Large NoC latency | Reduce per-hop latency<br>[Physical] Ongoing by colleague        |      |    |   |   |   |   |   |      |   |   |   |    |    |
|                   | Cut clk-domain (optional)<br>[RTL + Physical]                    |      |    |   |   |   |   |   |      |   |   |   |    |    |
|                   | NoC Topologies<br>[RTL + GVSoC + Physical]                       |      |    |   |   |   |   |   |      |   |   |   |    |    |

# Timeline



|                                 | 2025                                             |    |    |   |   |   |   |   |   | 2026 |   |   |    |    |  |
|---------------------------------|--------------------------------------------------|----|----|---|---|---|---|---|---|------|---|---|----|----|--|
|                                 |                                                  | 11 | 12 | 1 | 2 | 3 | 4 | 5 | 6 | 7    | 8 | 9 | 10 | 11 |  |
| Improve scalar core performance | Shared vector core [RTL + GVSoC]                 |    |    |   |   |   |   |   |   |      |   |   |    |    |  |
|                                 |                                                  |    |    |   |   |   |   |   |   |      |   |   |    |    |  |
| SW Kernel                       | Kernel optimization [SW] <b>Delayed, ongoing</b> |    |    |   |   |   |   |   |   |      |   |   |    |    |  |
|                                 | Kernel development [SW]                          |    |    |   |   |   |   |   |   |      |   |   |    |    |  |
| Report                          | Final Report and Benchmark                       |    |    |   |   |   |   |   |   |      |   |   |    |    |  |



# L1 Access Latency



- **Problem: increased L1 access latency when scaling up**
- **Solutions**
  - **(Ongoing, RTL & Physical) Hardware optimization**  
Further optimize the interconnection and cache controller to reduce access latency on existing architecture  
=> Retiming the current interconnection, remove unnecessary cuts in interco and cache  
=> Target to reduce 1-2 cycles (~5 cyc) for hit-to-use latency  
**Currently has reduced to 6 cycles:**  
**Core => Xbar => AMO => Ctrl (2) => AMO => Core**
  - **(Complete, RTL) Cache partitioning**  
Partition the shared L1 to keep data localized to avoid remote access latency
  - **(Complete with some problems, RTL & Physical) WT-L1**  
Explore and evaluate a private WT L1 for each core  
=> Performance degrade on single tile  
=> Needs more work to support larger system



# L1 Access Latency



- **Problem: large NoC latency (~28 cyc based on previous exploration)**
- **Solutions**
  - **(Ongoing, Physical) Reduce hop latency**  
Explore the timing under 7nm technology to see the possibility of reducing per-hop latency to 1 cycle  
=> Reduce the latency by 1/2
  - **(Planned, Physical) Cut clk-domain (optional)**  
If 1-cyc is not possible, explore to operate the NoC under a higher frequency than cores (e.g. 1.5GHz)  
=> Effectively cut the latency by 1/3
  - **(Planned, GVSoc & Physical) NoC Topology**  
Explore different NoC topologies  
=> Torus to reduce the diameter of NoC by 1/2, reduce longest latency by 1/2



# PE-Related



- **Problem: Improve scalar core performance**

- **Solutions**

- **(Planned, RTL & GVSoc) Shared vector core**

Explore two Snitch sharing one Spatz configuration to improve scalar performance

Software complexity needs to be considered, especially on the possible conflicting use of register file.

**Discussed Solution: critical section**



# RLC Workload



- **Problem: Current RLC model cannot reveal all properties of the kernel**
- **Solutions**
  - **(Ongoing, SW) Kernel optimization**  
Adapt the current model to better represent the real cases, e.g. do some operations on the data. Further cooperation with HQ side to develop a more complete and better model.
  - **(Planned, SW) Kernel development**  
Implement the missing use cases for multi-user





# Thank you!

## Q&A

